

Logging into (and out of) Linux Systems

A **Shell** provides you with an interface to the Unix system. It gathers input from you and executes programs based on that input. When a program finishes executing, it displays that program's output.

Shell is an environment in which we can run our commands, programs, and shell scripts. There are different flavours of a shell, just as there are different flavours of operating systems. Each flavour of shell has its own set of recognized commands and functions.

In Linux, there are two ways to log in.

1) Text-based (TTY) terminals:

When you connect to a LINUX computer remotely (using telnet) or when you log in locally using a text-only terminal, you will see the prompt:

```
login:
```

At this prompt, type in your username and press the enter/return/  key. Remember that LINUX is case sensitive (i.e. Will, WILL and will are all different logins). You should then be prompted for your password:

```
login: will
password:
```

Type your password in at the prompt and press the enter/return/  key. Note that your password will not be displayed on the screen as you type it in.

If you mistype your username or password you will get an appropriate message from the computer and you will be presented with the `login:` prompt again. Otherwise you should be presented with a shell prompt which looks something like this:

```
$
```

To log out of a text-based LINUX shell, type "exit" at the shell prompt (or if that doesn't work try "logout"; if that doesn't work press ctrl-d).

2) Graphical terminals:

If you're logging into a LINUX computer locally, or if you are using a remote login facility that supports graphics, you might instead be presented with a graphical prompt with login and password fields. Enter your user name and password in the

same way as above (N.B. you may need to press the TAB key to move between fields).

Once you are logged in, you should be presented with a graphical window manager that looks similar to the Microsoft Windows interface. To bring up a window containing a shell prompt look for menus or icons which mention the words "shell", "xterm", "console" or "terminal emulator".

To log out of a graphical window manager, look for menu options similar to "Log out" or "Exit".

Concept of Shell

A login shell is a shell given to a user upon login into their user account. This is initiated by using the `-l` or `--login` option, or placing a dash as the initial character of the command name, for example invoking bash as `-bash`.

- Whenever you login to a Linux system you are placed in a program called the shell. All of your work is done within the shell.
- The shell is your interface to the operating system. It acts as a command interpreter; it takes each command and passes it to the operating system. It then displays the results of this operation on your screen.

Various Linux Shell

There are several shells in widespread use. The most common ones are described below.

Shell Types

In Unix, there are two major types of shells –

Bourne shell – If you are using a Bourne-type shell, the `$` character is the default prompt.

C shell – If you are using a C-type shell, the `%` character is the default prompt.

The **Bourne Shell** has the following subcategories –

- Bourne shell (sh)
- Korn shell (ksh)
- Bourne Again shell (bash)
- POSIX shell (sh)

The different **C-type shells** follow –

- C shell (csh)
- TENEX/TOPS C shell (tcsh)

Bourne shell (sh)

Original Linux shell written by Steve Bourne of Bell Labs. Available on all LINUX systems. Does not have the interactive facilities provided by modern shells such as the C shell and Korn shell.

The Bourne shell does provide an easy to use language with which you can write shell scripts.

Korn shell (ksh)

Written by David Korn of bell labs. It is now provided as the standard shell on Linux systems.

Provides all the features of the C and TC shells together with a shell programming language similar to that of the original Bourne shell.

Bourne Again Shell (bash)

Public domain shell written by the Free Software Foundation under their GNU initiative. Ultimately it is intended to be a full implementation of the IEEE Posix Shell and Tools specification.

Widely used within the academic community. Provides all the interactive features of the C shell (csh) and the Korn shell (ksh). Its programming language is compatible with the Bourne shell (sh).

POSIX shell (sh)

POSIX is an acronym for “Portable Operating System Interface”. POSIX shell is based on the standard defined in Portable Operating System Interface (POSIX) – IEEE P1003.2.

It is a set of standards codified by the IEEE and issued by ANSI and ISO. POSIX makes task of cross-platform software development easy.

There are various POSIX versions, but the most important are POSIX.1 and POSIX.2, which define system calls and command-line interface.

C shell (csh)

Written at the University of California, Berkley. As its name indicates, it provides a C like language with which to write shell scripts.

TC Shell (tcsh)

Available in the public domain. It provides all the features of the C shell together with EMACS style editing of the command line.

Your login shell is usually established by the local System Administrator when your userid is created. You can determine your login shell with the command:

```
echo $SHELL
```

- Each shell has a default prompt. For the 5 most common shells:

```
$ (dollar sign)      - sh, ksh, bash
```

```
% (percent sign)     - csh, tcsh
```

Linux Shells and their Features

Depending upon the shell, certain features will be available. The table below summarizes the features available with the 5 most common shells. Note: these features will be described in detail later.

	Bourne	C	TC	Korn	BASH
	sh	csch	tcsh	ksh	bash
Programming language	Yes	Yes	Yes	Yes	Yes
Shell variables	Yes	Yes	Yes	Yes	Yes
Command alias	No	Yes	Yes	Yes	Yes
Command history	No	Yes	Yes	Yes	Yes
Filename completion	No	Yes*	Yes	Yes*	Yes
Command line editing	No	No	Yes	Yes*	Yes
Job control	No	Yes	Yes	Yes	Yes

* not the default setting for this shell

The primary advantages of interfacing to the system through a shell are as follows (**Features**):

1) Wildcard substitution in file names (pattern-matching)

Carries out commands on a group of files by specifying a pattern to match, rather than specifying an actual file name.

2) Background processing

Sets up lengthy tasks to run in the background, freeing the terminal for concurrent interactive processing.

3) Command aliasing

Gives an alias name to a command or phrase. When the shell encounters an alias on the command line or in a shell script, it substitutes the text to which the alias refers.

4) Command history

Records the commands you enter in a history file. You can use this file to easily access, modify, and reissue any listed command.

5) File name substitution

Automatically produces a list of file names on a command line using pattern-matching characters.

6) Input and output redirection

Redirects input away from the keyboard and redirects output to a file or device other than the terminal. For example, input to a program can be provided from a file and redirected to the printer or to another file.

7) Piping

Links any number of commands together to form a complex program. The standard output of one program becomes the standard input of the next.

8) Shell variable substitution

Stores data in user-defined variables and predefined shell variables.

You may refer following link for more information

<https://www.ibm.com/docs/en/aix/7.2?topic=shells-shell-concepts>

Shell	Complete path-name	Prompt for root user	Prompt for non root user
Bourne shell (sh)	/bin/sh and /sbin/sh	#	\$
GNU Bourne-Again shell (bash)	/bin/bash	bash-VersionNumber#	bash-VersionNumber\$
C shell (csh)	/bin/csh	#	%
Korn shell (ksh)	/bin/ksh	#	\$
Z Shell (zsh)	/bin/zsh	<hostname>#	<hostname>%

You may refer following link for more information

<https://www.journaldev.com/39194/different-types-of-shells-in-linux>